Part 17

CRUD Operation in Django

VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

CRUD Operation

CRUD stands for Create, Read, Update & Delete. These are the four basic operations which are executed on Database Models. We are developing a web app which is capable of performing these operations.

1. Read Operation
The ability of the application to read data from the database.

2. Create Operation
The ability of the application to store data in the database.

3. Update Operation
The ability of the application to edit the stored value in the database.

4. Delete Operation
The ability of the application to delete the value in the database.

1. Making a Library CRUD Application

Our first steps would be to make a new project in Django and a new virtual environment. To make a virtual environment, enter:

Now, install the necessary software in this environment. Once, the environment is set up, activate the same. After activating the environment, install django on it.

pip install django==2.1.7

2. Setting Django Project

After the installation is complete make new Django project.

1) Create project name “libraryapp”
2) Create app for project name “book”
3) Installing the application and other important settings
4) Setup static file and media file in settings.py

```
# settings for Static files
STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'static-files')

# settings for Media files
MEDIA_URL = 'media/'
MEDIA_ROOT=os.path.join(BASE_DIR, 'media')
```

Pillow Package

Above setting is important for saving and serving static files to the client. Also, we need a pillow package of Python. Install it via command line in your virtual environment.

pip install pillow

Pillow is a Python Imaging Library which deals with different image files. We won't be using pillow directly but Django is using it. Therefore, we need pillow library.

3. Making Models for Books App

In the book folder, open models.py and paste this code. We are making a model or database table for our books app.

```
from django.db import models
#DataFlair Models
class Book(models.Model):
    name = models.CharField(max_length = 50)
    picture = models.ImageField()
    author = models.CharField(max_length = 30, default='anonymous')
    email = models.EmailField(blank = True)
    describe = models.TextField(default = 'DataFlair Django tutorials')
    def __str__(self):
        return self.name
```

4. Making Model Forms in Book Directory

Django makes it so much easier to make forms for models. We just need to use our models and we can easily make forms.

```
from django import forms
from .models import Book

# Create form for model
class BookCreate(forms.ModelForm):
    class Meta:
        model = Book
        fields = '__all__'
```

5. Registering Model in Django Admin

Here we are editing **admin.py** existing in **book** folder. Import the model you want to register in the admin. In this case, it is a Book.

```
# Register the book model
admin.site.register(Book)
```

Okay, so we have made a lot of backend here. To implement all of this, run these commands in the command line:

Migrate and create super user

>>> python manage.py makemigrations

>>> python manage.py migrate

After running these commands, we also need a superuser to login to the admin. You can make a superuser using the command:

>>> python manage.py createsuperuser

6. Making View Functions for Django CRUD App

The view functions are our actual CRUD operations in Django. Now, we are editing **views.py** in **book** folder. Open **views.py** file in the folder. Paste this code in it:

```
from django.shortcuts import render, redirect
from .models import Book
from .forms import BookCreate
from django.http import HttpResponse

# Get all books data in the database
def index(request):
    shelf = Book.objects.all()
    return render(request, 'book/library.html', {'shelf': shelf})
```

Book Form Data save in Database

```
from django.shortcuts import render, redirect
from .models import Book
from .forms import BookCreate
from django.http import HttpResponse

def upload(request):
    upload = BookCreate()
    if request.method == 'POST':
        upload = BookCreate(request.POST, request.FILES)
        if upload.is_valid():
            upload.save()
            return redirect('index')
        else:
            return HttpResponse("""your form is wrong,<a href = "{{ url : 'index'}}">reload</a>""")
else:
        return render(request, 'book/upload_form.html', {'upload_form':upload})
```

Update the Book Form Data

```
def update_book(request, book_id):
    book_id = int(book_id)
    try:
        book_sel = Book.objects.get(id = book_id)
    except Book.DoesNotExist:
        return redirect('index')
    book_form = BookCreate(request.POST or None, instance = book_sel)
    if book_form.is_valid():
       book_form.save()
       return redirect('index')
    return render(request, 'book/upload_form.html', {'upload_form':book_form})
```

Delete Book Data

```
from django.shortcuts import render, redirect
from .models import Book
from .forms import BookCreate
from django.http import HttpResponse

def delete_book(request, book_id):
    book_id = int(book_id)
    try:
        book_sel = Book.objects.get(id = book_id)
    except Book.DoesNotExist:
        return redirect('index')
    book_sel.delete()
    return redirect('index')
```

7. Making Templates

The first thing you need to do is to make the **templates** folder in the **book** folder. Inside **book/templates**, make another folder **book**. We are going to make all our templates in that folder.

```
{% block content %}
<div class="card-columns">
    {% for book in shelf %}
        <img class="card-img-top" src="{{ book.picture.url }}">
        <h5 class="card-title">{{ book.name }}</h5>
        <p class="card-text">{{ book.describe }}</p>
        <p class="card-title">{{book.author}}</p>
   <center>
       <a href="update/{{book.id}}" class="btn btn-warning" id = '{{book.id}}'>edit</a>
       <a href="delete/{{book.id}}" class="btn btn-danger" id = '{{book.id}}'>delete</a>
    </center>
    {% endfor %}
{% endblock %}
```

Upload.html

```
{% block content %}
    <center>
    <h1 class="display-3">Upload Books</h1>
    <form method = 'POST' enctype="multipart/form-data">
        {% csrf_token %}
        <table class = 'w-50 table table-light'>
            {% for field in upload_form %}
            <tr>
                <th>{{field.label}}</th>
                <td>{{ field }}</td>
            </tr>
            {% endfor %}
        </table>
        <button type="submit">Submit</button>
    </form>
</center>
{% endblock %}
```

8. Configuring URLs

Now, we need to configure the urls file. Paste this code as it is in the mentioned urls files. If the file doesn't exist then make one and copy the whole thing.

```
from django.contrib import admin
from django.urls import path, include
#DataFlair
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('book.urls'))
]
```

book/urls.py

```
from django.urls import path
from . import views
from libraryapp.settings import DEBUG, STATIC_URL, STATIC_ROOT, MEDIA_URL, MEDIA_ROOT
from django.conf.urls.static import static

urlpatterns = [
    path('', views.index, name = 'index'),
    path('upload/', views.upload, name = 'upload-book'),
    path('update/<int:book_id>', views.update_book),
    path('delete/<int:book_id>', views.delete_book)
]

#DataFlair
if DEBUG:
    urlpatterns += static(STATIC_URL, document_root = STATIC_ROOT)
    urlpatterns += static(MEDIA_URL, document_root = MEDIA_ROOT)
```

9. Running Server and Testing

At last, the fun part. To test the website, start your server by:

>>> python manage.py runserver

Thanks for Reading